

Week 7: Building the Frontend: React 3

Problem Statement

This week you will complete the frontend for **ThreadHive**, a social media platform where users can create communities (subreddits), post discussion topics (threads), add comments, vote and interact with content.

In this assignment, you will wire up React Router with protected routes, implement the navigation and authentication flow, fetch threads from the API, and use GitHub Copilot to build a Create Thread form, perform automated testing with Playwright CLI, and build a full Profile page.

Project Setup and Structure

1. Download and extract the assignment starter code from the zip file provided on Olympus.
2. The project includes code for the backend and the frontend. You do not need to modify the backend for this assignment, but you will need to have it running locally to test the frontend features you build. Open the backend and the frontend folders in separate VS Code windows. Since we will be making changes only to the frontend code, we can isolate it in its own VS Code window to ensure that the agent is not fed unnecessary context.
3. To get the backend running, in the VS Code window where you have the backend folder open, follow these steps:
 - Open a terminal in the backend folder.
 - Install dependencies: `npm install`
 - Populate the database: `npm run populate`
 - Start the server: `npm run dev`

Ensure that you have set up the .env file appropriately for the backend.

4. Open the VS Code window where you have the frontend folder open.
 - Initialize a new Git repository (either by running `git init` in the terminal or by using the VS Code Git extension).
 - As you make changes throughout this tutorial, remember to regularly commit your work.

The main parts of the frontend codebase are as follows:

src/	
├─ App.jsx	Root component with routing and auth provider
├─ main.jsx	Entry point that renders the App component
├─ api/	
│ └─ apiClient.js	Shared API client utility (fetchAPI)

```

├── components/           Reusable UI components (Comment, ThreadList,
Forms, Shared, Header, Footer, PrivateRoute)
├── config/
│   └── apiConfig.js      API endpoint configuration
├── context/              Auth context provider (AuthContext.jsx)
├── pages/                Page components (Auth, User)
├── services/             API service modules (authService,
threadService, commentService, subredditService, userService)
└── utils/
    └── handleApiError.js Error handling utility

```

This session is structured in two parts:

	Part	Focus	AI Usage
Part 1	Manual Implementation	Routing, Header Navigation, Login, Load Threads, Thread Service	No AI Tools
Part 2	AI-Assisted Development	Custom Instructions Update, Create Thread Form, Profile Page	GitHub Copilot

In **Part 1** you will wire up the app's routing and authentication flow manually, without AI assistance. This ensures you understand how protected routes, localStorage-based auth state, and API service functions work together.

In **Part 2** we will use GitHub Copilot to implement additional features, including a Create Thread form and a Profile page, and to perform automated testing with Playwright CLI.

By the end of this session, you will be able to:

- Implement protected routes in React Router
- Manage authentication state with React Context and localStorage
- Fetch and display data from an API in React
- Use GitHub Copilot to generate React components based on specific requirements
- Use Playwright CLI to automate testing of browser flows

Part 1: Manual Implementation

In this part, you will complete the core routing, navigation, authentication, and data-fetching logic for ThreadHive frontend without using any AI tools.

1.1 Project Setup

1. Open a terminal in the frontend folder.
2. Install the dependencies: `npm install`
3. Start the development server to confirm everything is working: `npm run dev`

Open `http://localhost:5173` in your browser. You should see the app shell (header and footer) but no page content yet — that is expected until you add the routes.

1.2 Wrap Auth Provider, Add Routes (`App.jsx`)

a. Wrap the `<BrowserRouter>` in `App.jsx` with the `AuthProvider` component from `AuthContext` to provide auth state to the entire app.

b. Add the routes for the app in `App.jsx` inside the `<Routes>` block:

- `/login` → renders the `Login` component
- `/register` → renders the `Register` component
- `/home` → renders the `Home` component, **wrapped in `<PrivateRoute>`**
- `/thread/:id` → renders the `ThreadPage` component, **wrapped in `<PrivateRoute>`**

The catch-all route (`/*` → redirect to `/home`) is already provided — leave it as-is.

Use the `<PrivateRoute>` component to protect authenticated routes.

Once done, navigating to `http://localhost:5173/login` should render the Login page.

1.3 Add Header Navigation (`src/components/Header/Header.jsx`)

Review the 'AuthContext' already implemented in 'src/contexts/AuthContext.jsx'. It provides the current auth state (token and user) as well as login and logout functions.

In the `Header` component read the `token` and `user` from `AuthContext` via `useAuth()`. Three handler functions have empty bodies — implement each one:

- `handleLogin` — navigate to `/login`
- `handleSignup` — navigate to `/register`
- `handleLogout` — call `logout()` from `AuthContext` (add it to the `useAuth()` destructure at the top of the component), then navigate to `/login`

After completing this task, clicking **Login** and **Register** in the header should navigate to the correct pages.

1.4 Write Authentication Logic (`src/pages/Auth/Login.jsx`)

Implement the `handleSubmit` function in `Login.jsx` to perform the login flow. Call the `login` function from `authService.js` to authenticate the user, on success call `loginUser` function from the `AuthContext` to store auth state, then navigate to `/`.

You can test the login flow: create a user via the registration page, then log in with that user's credentials. Upon successful login, you should be redirected to the home page. There will be no threads displayed yet — that is expected until you implement the thread loading logic in the next two steps.

1.5 Load Threads (`src/pages/User/Home.jsx`)

Implement the `loadThreads` async function inside the `useEffect` in `Home.jsx` to fetch and display the list of recent threads. Use the `fetchRecentThreads` function from `threadService.js` to call the API and retrieve the threads. Make sure to handle loading and error states appropriately.

The threads will still not display after this step because the service functions are not implemented yet — that is expected until you complete the next step.

1.6 Implement Thread Service (`src/services/threadService.js`)

Implement all the service functions in `threadService.js` to call the corresponding API endpoints using the `fetchAPI` utility.

After completing this task, the home feed should populate with the list of threads in the database.

Part 2: AI-Assisted Development with GitHub Copilot

In this part, you will use GitHub Copilot to implement additional features in the application. Before writing any code with Copilot, let us first prepare the custom instructions file to reflect the conventions and patterns that are now established in the codebase.

2.1 Generate Custom Instructions

Generate custom instructions (a `AGENTS.md` file) using the `/init` command.

Carefully review the generated instructions and edit them if necessary. Ensure that they include instructions on the use of Context API for auth state management and the use of the `fetchAPI` utility for making API calls in service functions. This will help guide Copilot to generate code that is consistent with the existing codebase and architectural patterns.

2.2 Create Thread Form (`src/pages/User/Home.jsx`)

The `CreateThreadForm` component is already built and available at `src/components/Forms/CreateThreadForm.jsx`. Your task is to wire it into `Home.jsx` so that a **"Create Thread"** button appears on the home page, and clicking it toggles the form in and out of view on the same page.

Open a new Copilot Chat and switch to **Agent Mode**. Use the following prompt:

```
In src/pages/User/Home.jsx, add a "Create Thread" button that toggles a form into view.
```

Requirements:

- Add a boolean state variable called `showForm`, initialized to `false`
- Add a **" Create"** button in the heading row of the home feed, aligned to the right of the **" Home Feed"** heading

- When showForm is true, render the CreateThreadForm component (from src/components/Forms/CreateThreadForm.jsx) below the heading row
- Pass an onClose prop to CreateThreadForm that sets showForm back to false
- When showForm is true, change the button label to "× Close"
- The button should toggle showForm on each click

Review the generated code carefully before accepting it. Iterate with Copilot if any of these points are missing or incorrect.

2.3 Build Profile Page (````)

In src/pages/User/Home.jsx, add a "Create Thread" button that toggles a form into view.

Requirements:

- Add a boolean state variable called showForm, initialized to false
- Add a "+ Create" button in the heading row of the home feed, aligned to the right of the "Home Feed" heading
- When showForm is true, render the CreateThreadForm component (from src/components/Forms/CreateThreadForm.jsx) below the heading row
- Pass an onClose prop to CreateThreadForm that sets showForm back to false
- When showForm is true, change the button label to "× Close"
- The button should toggle showForm on each click

Review the generated code carefully before accepting it. Iterate with Copilot if any of these points are missing or incorrect.

2.3 Build Profile Page (src/pages/User/Profile.jsx)

The Profile.jsx file exists in src/pages/User/ but is currently empty. You will use Copilot's **Agent Mode** to build the entire component from scratch, using the `AuthContext` to read and update the user.

Open a new Copilot Chat and switch to **Agent Mode**. Use the following prompt:

Build the Profile component in src/pages/User/Profile.jsx.

Requirements:

- Import and use the `useAuth()` hook from `src/context/AuthContext.jsx` to get the current user and the `updateUser` function: `const { user, updateUser } = useAuth();`
- Display the user's name, email, bio, location, and website

- Show an avatar circle with the user's first initial (uppercase); fall back to "U" if name is not set
- Add a "← Back to Home" button that uses `useNavigate()` to navigate to `/home`
- Add an "✎ Edit Profile" toggle button that shows and hides an edit form
- In view mode, display each field (name, email, bio, location, website) with a label
- In edit mode, show form inputs for name, bio, location, and website — the email field must be present but disabled
- On save: call `updateUser({ ...user, ...form })` to persist the changes, then set editing to false
- On cancel: set editing to false without saving
- Add a `useEffect` that syncs the form state when the user object changes (so the displayed values update after a save)
- Use the CSS classes already defined in `Profile.css`

Review the generated code carefully before accepting it. Iterate with Copilot if any of these points are missing or incorrect. Pay particular attention to the ```` Build the Profile component in `src/pages/User/Profile.jsx`.

Requirements:

- Import and use the `useAuth()` hook from `src/context/AuthContext.jsx` to get the current user and the `updateUser` function:
`const { user, updateUser } = useAuth();`
- Display the user's name, email, bio, location, and website
- Show an avatar circle with the user's first initial (uppercase); fall back to "U" if name is not set
- Add a "← Back to Home" button that uses `useNavigate()` to navigate to `/home`
- Add an "✎ Edit Profile" toggle button that shows and hides an edit form
- In view mode, display each field (name, email, bio, location, website) with a label
- In edit mode, show form inputs for name, bio, location, and website – the email field must be present but disabled
- On save: call `updateUser({ ...user, ...form })` to persist the changes, then set editing to false
- On cancel: set editing to false without saving
- Add a `useEffect` that syncs the form state when the user object changes (so the displayed values update after a save)
- Use the CSS classes already defined in `Profile.css`

Review the generated code carefully before accepting it. Iterate with Copilot if any of these points are missing or incorrect. Pay particular attention to the `updateUser` call — Copilot may try to write to `localStorage` directly. If it does, remind it of the custom instruction: *"use `updateUser()` from `useAuth()` to save profile changes."*

2.4 Automated Testing with Playwright CLI

Playwright CLI is a command-line tool for browser automation designed to work efficiently with AI coding agents. Unlike the Playwright test framework, Playwright CLI is invoked directly from the terminal (or by an agent), making it a lightweight way to automate and verify browser flows without writing test files.

Install Playwright CLI

Install the package globally so it is available from any terminal session:

```
npm install -g @playwright/cli@latest
```

Then install the browser binaries and register the CLI skills for agent use:

```
playwright-cli install --skills
```

Verify the installation:

```
playwright-cli --help
```

Test the Authentication Flow

Make sure your development server is running (``bash npm install -g @playwright/cli@latest

```
Then install the browser binaries and register the CLI skills for agent use:
```

```
``bash  
playwright-cli install --skills
```

Verify the installation:

```
playwright-cli --help
```

Test the Authentication Flow

Make sure your development server is running (**npm run dev**) and the backend API is available, then open a Copilot Chat in **Agent Mode** and use the following prompt:

Using playwright-cli, test the full authentication flow for the ThreadHive app running at `http://localhost:5173`.

Test that a new user can register, log in, and is redirected to the home page upon successful login. Then test that the user can log out and is redirected to the login page.

Observe how the agent tests the authentication flow by launching a browser, interacting with the UI, and verifying the expected outcomes at each step. This is a powerful way to validate that your authentication flow works as intended from the user's perspective.